

	TIPO DOCUMENTO	
Software life cycle guide		Rev. draft Data 16/12/2025
		Pagina 1 di 6

STORIA DELLE VERSIONI

Rev	Date	Document	Issued by	Checked by	Approved by
1	16/12/2025		SB		
2	14/05/2026		SB		

DOCUMENTI RICEVUTI

Titolo	Descrizione	Rev	Data	Ricevuto Tramite

DOCUMENTI ALLEGATI

Titolo	Descrizione	Rev	Data	Inviato Tramite

	TIPO DOCUMENTO		
Software life cycle guide		Rev. draft Data 16/12/2025	Pagina 2 di 6

INDICE

Indice.....	2
1 Scopo.....	3
2 Campo di applicazione.....	3
3 Definizioni ed abbreviazioni.....	3
4 Struttura.....	3
4.1 Gestione bugfixing.....	4
4.2 Versioning.....	5
4.3 Release note.....	5
5 Esempio.....	5

	TIPO DOCUMENTO		
Software life cycle guide		Rev. draft Data 16/12/2025	Pagina 3 di 6

1 Scopo

Lo scopo di questo documento è definire le regole per la gestione dei repository aziendali e le relative strutture.

2 Campo di applicazione

Sviluppo codice, software lifecycle, quality assurance, pubblicazione

3 Definizioni ed abbreviazioni

- Repository: spazio di archiviazione dei progetti aziendali, sottoposto a controllo di versione
- Branch: divisione logica dello spazio di archiviazione. Rappresenta una "copia" del codice in un dato momento dello sviluppo
- Commit: azione di salvataggio dello stato del codice sul proprio computer
- Push: sincronizzazione degli stati del codice tra il proprio computer e il server
- Pull: sincronizzazione degli stati del codice tra il server e il proprio computer
- Pull Request (o PR): azione di sincronizzazione tra due branch
- Github: gestore di repository
- Github Branch protection rule: regole per la pubblicazione su un dato branch
- Github Workflow: automazione fornita da Github, permette di scrivere plugin che validano il codice ad un determinato momento
- Github Action: ambiente di esecuzione di un Github workflow

4 Struttura

Ogni repository aziendale presenta 3 principali branches:

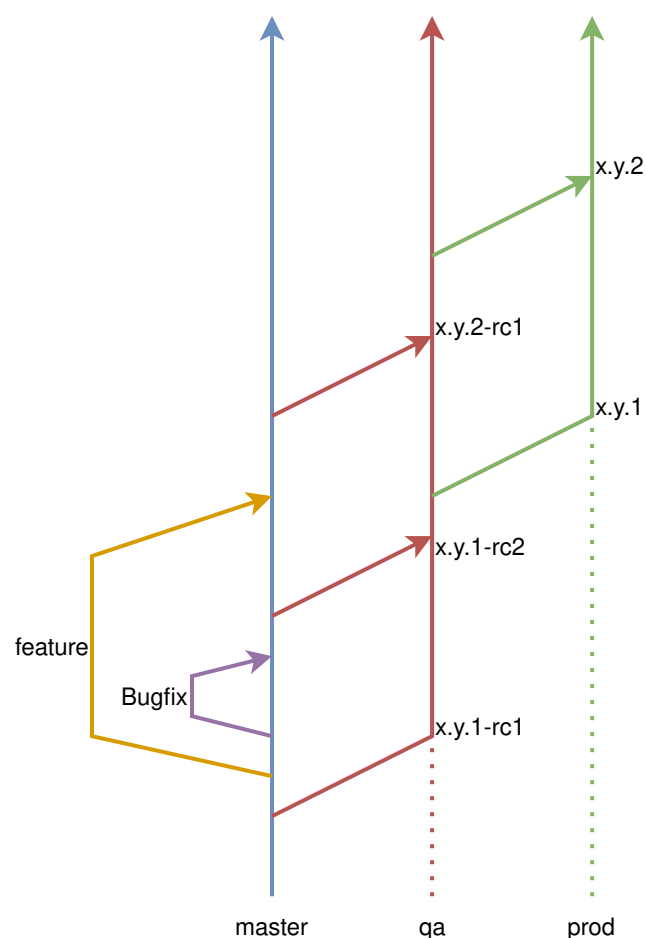
- master (o main): è il branch che rappresenta lo stato di sviluppo più aggiornato
- qa: è il branch che rappresenta lo stato del codice in test
- prod: è il branch che rappresenta lo stato del codice pubblicato

Il flow dello sviluppo segue lo schema:

1. Data una feature da implementare lo sviluppatore esegue un Pull per allineare le ultime modifiche presenti
2. Crea un branch chiamato "feature/<nome_della_feature_da_sviluppare>" o "bugfix/<nome_del_bug_da_risolvere>"
3. Completa il ciclo di sviluppo eseguendo commit continui e descrittivi, il più atomici possibile
4. Crea una Pull Request tra la sua branch di sviluppo e il branch principale (main o master)

	TIPO DOCUMENTO		
Software life cycle guide		Rev. draft Data 16/12/2025	Pagina 4 di 6

5. Un programmatore terzo verifica e approva la PR
6. il codice viene allineato tra il branch di sviluppo e il branch principale
7. Quando vengono approvate un dato numero di feature/bugfix, viene creata una Pull Request tra il branch principale e il branch di QA
8. I tester verificano il codice
 1. Se si trovano bug vengono aperti nuovi branch di bugfixing
9. Viene creata una Pull Request tra il branch di QA e il branch di PROD
10. Vengono eseguite le funzionalità di distribuzione automatica del codice tramite Github Action



4.1 Licenza

Ogni file creato all'interno del progetto deve contenere, nella parte iniziale un header che descrive la licenza del file stesso, e per estensione, la licenza del progetto.

	TIPO DOCUMENTO	
Software life cycle guide		Rev. draft Data 16/12/2025
		Pagina 5 di 6

Si allega la versione dell'header compatibile con il linguaggio python. Potrebbe essere necessario modificarla a seconda della modalità di inserimento dei commenti nel linguaggio utilizzato.

La licenza deve essere compilata nelle parti del:

- Project Name – nome del progetto
- Project Number – numero del progetto, come segnato in vtiger
- Release date – data della release (se applicabile)
- Release version – versione della release (se applicabile)
- Contributors – sviluppatori che hanno partecipato allo sviluppo del progetto

```

*****
***
# Copyright (c) 2026 Next Industries s.r.l.
#
# This program and the accompanying materials are made available under the
# terms of the Apache 2.0 which is available at
# http://www.apache.org/licenses/LICENSE-2.0
#
# SPDX-License-Identifier: Apache-2.0
#
# Project Name:
#
# Project Number:
#
#
# Release date:
# Release version:
#
# Contributors:
# - Massimiliano Bellino
# - Stefano Barbareschi
*****
***

```

4.2 Gestione bugfixing

La gestione dei bug segue il flow di pubblicazione del repository. Quando viene individuato un bug e uno sviluppatore ne prende in carico la risoluzione, crea un branch, chiamato "bugfix/<nome del bug>" dal branch principale. Questo branch viene poi integrato nel branch principale e da lì viene creata una nuova PR per passare in QA.

4.3 Versioning

Lo schema di versionamento deve avere *almeno* 3 numeri:

- Major release number
- Middle release number
- Minor release number

	TIPO DOCUMENTO		
Software life cycle guide		Rev. draft Data 16/12/2025	Pagina 6 di 6

Se la versione viene distribuita in QA deve avere il suffisso rc<numero sequenziale>, che sta ad indicare che questa è una **Release Candidate** ovvero una possibile release in PROD.

4.4 Release note

Ogni volta che viene fatta una nuova release in QA o in PROD, deve essere emessa una release note che contiene:

- Release date: data di release
- Release number: numero di release (con il suffisso rc<numero> per il branch di QA). Il suffisso rc significa Release Candidate, ad indicare una possibile release ma non ancora validata
- Feature e bugfixing: un elenco dei cambiamenti inclusi nella release

5 Esempio

Viene mostrato l'esempio del progetto Tactigon Shapes. Il repository prevede 2 workflow e 3 Branch protection Rules:

- Workflow "branch-flow-check": verifica che possano essere aperte PR solo tra il branch QA e master, PROD e QA. Questo per garantire che il software lifecycle segua la struttura descritta in questo documento
- Workflow "docker-publish": viene lanciato ad ogni commit in PROD e distribuisce il codice su docker-hub
- Branch protection rules: sui branch di master, qa e prod è possibile solo scrivere tramite PR e non direttamente tramite commit

Il versioning di questo progetto è composto da 4 numeri:

- Major version number HW Tactigon Skin
- Middle version number SW Tactigon Gear
- Middle version number SW Tactigon Speech
- Minor version number Tactigon Shape

es:

- HW Tactigon Skin: 5
- Tactigon Gear: 5.4.2
- Tactigon Speech: 5.0.10
- Tactigon Shapes: 2

La versione completa in QA sarà 5.4.0.2-rc1.

La versione pubblicata in PROD sarà 5.4.0.2.